Assignment 1: Single-Server Key-Value Store

CSCI 5144 – Cloud Computing · https://github.com/Laeeq14/CSCI_5144_Assignments/

Laeeq Ahmed fazil017@umn.edu
Adam Meziou mezio004@umn.edu
Dang Nguyen nguy6082@umn.edu
Yassir Abdalla abdal167@umn.edu
Younus Ali ali00380@umn.edu

Design Decisions

The server is written in Go with the standard `net/http` package, chosen for goroutine-based concurrency, built-in HTTP keep-alive, and a single static binary with no runtime dependency.

Concurrency. A `map[string]string` is guarded by a `sync.RWMutex`: GETs take a read lock and run in parallel; PUT and DEL take the write lock, so concurrent writes to the same key are serialized. PUT is exposed as `POST /<key>` with a JSON body, matching the provided benchmark.

Persistence. The store is flushed every 10 s and within 2 s of any write (fixed-deadline debounce). Each save copies the map under a read lock, writes a temp file, calls `fsync`, and atomically renames it. On Ctrl+C (SIGINT) a final flush runs before exit; tested by `kv_bench.py set`, Ctrl+C, restart, `get` (`mismatch=0`).

Logging. Operations go through a 65,536-entry buffered channel to a background writer (flushed every 200 ms), keeping file I/O off the request path. Timestamps are taken in the request goroutine. If the channel fills, entries are dropped and counted (reported at shutdown); no drops were observed in any benchmark run.

Error handling. Missing key: 404; malformed JSON or missing `value`: 400; body over 1 MiB: 413 (all with a JSON error body); unsupported method: 405 (plain-text body).

Challenges

The main challenge was separating client from server bottlenecks. The provided benchmark uses three Python threads sharing the GIL, and on our machine `requests` costs about 0.37 ms of CPU per call (measured with a single `http.client` connection against a warm server), so throughput is client-bound at roughly 2,000–3,000 ops/s regardless of server speed. Running the same benchmark under WSL2 gave a similar score, which supports the client-bound explanation rather than a Windows-specific effect. A Go load generator reached about 169k ops/s at 100 goroutines (non-verifying Go load generator on the same host, peak of the sweep); see README. A second issue was shutdown: waiting for the persistence goroutine with `time.Sleep` could cut off a slow `fsync`, so it was replaced with a done channel that `main` blocks on.

Testing and Results

Windows 11, Go 1.27.0, Python 3.14.4, laptop plugged in. `kv_bench.py` reported `mismatch=0, error=0` in both phases, and data survived a Ctrl+C restart. Eight runs gave 2,062–2,221 ops/s (median 2,101, average latency about 1.4 ms). On battery the same benchmark gave 897–1,144 ops/s, consistent with CPU throttling of the Python client. An 18-case edge test suite (missing keys, bad or oversized bodies, special characters, non-ASCII values, wrong methods) passes on every run.

Assumptions

Single process, no replication. A hard kill within the 2 s debounce window can lose recent writes. Bodies are limited to 1 MiB; keys and values are UTF-8 strings. Benchmark results use the `kv_bench.py` script provided by the instructor with its default configuration (3 threads, 300 keys).

Potential Improvements

A write-ahead log would remove the loss window without a full snapshot per write; sharding the map with per-shard locks would reduce write contention; TTLs would support cache workloads; and Raft-based replication would add fault tolerance.